

PYCON US 2026 - 30 MIN

Breaking the Speed Limit

Fast statistical models with Python 3.14, Numba, and JAX
- told from the workflow of a statistician.

Wenxin Jiang and Jian Yin - Department of Biostatistics, City University of Hong Kong

Not a speed contest

The question is not which logo wins. The question is which implementation preserves the statistic and removes the bottleneck.

Simulation first

When real data has no answer key, simulation gives us one.

Clear -> fast -> scalable

Reference code, stress cases, then Python acceleration tools.

From a statistician's point of view, speed is only useful after trust

We use Python to move through the whole scientific loop: write the model, test the behavior, find hard cases, then accelerate the expensive pattern.

1. The method is still being learned

In practice, the exact statistical specification often becomes clearer through simulation: which assumptions matter, which cases fail, which outputs should be stable.

2. The reference is part of the science

Readable Python or NumPy code is not throwaway. It is the executable definition that optimized kernels must match.

3. Acceleration must preserve meaning

Numba, JAX, threads, and GPUs are useful only when they compute the same statistic on the same controlled scenarios.

Rule for the talk: a faster implementation that changes the statistic is not an optimization. It is a different method.

Simulation creates the answer key that real data does not give us

Known target

True clusters, null behavior, effect size, invariants, expected calibration, or exact reference output.

Known stress cases

Low separation, imbalance, outliers, high dimension, bad seeds, memory pressure, and worker overhead.

Known decision rule

Only variants that pass the statistical contract enter the runtime comparison.

STEP 1

Reference

Write code close to the math, even if it is slow.

STEP 2

Scenarios

Generate easy, null, alternative, and pathological cases.

STEP 3

Validate

Check recovery, calibration, equivalence, and numerical stability.

STEP 4

Optimize

Choose the smallest tool that removes the proven bottleneck.

Simulation-driven statistical computing is statistical CI

The testing instinct is familiar; the oracle is statistical behavior rather than one expected scalar.

Software testing term	Statistical simulation equivalent
Unit tests	Fixed-seed reference equivalence
Property tests	Null-model invariants, symmetry, monotonicity, calibration
Fuzz/stress tests	Imbalance, outliers, bad seeds, high dimension, memory risk
Load tests	Scale N, d, K, p, R, seeds, workers, and batch sizes
Regression tests	Optimized paths preserve the same statistic after code changes

Python is the control plane; the hotspot can move to the right engine

The point is not to make every line of Python magically fast. The point is to keep the model readable while moving the expensive computational pattern into NumPy, Numba, threads, or JAX when the simulation contract allows it.

Tool	Best role in this workflow	What to watch
Python / NumPy reference	Executable statistical definition; easiest to inspect and debug	Loops and large temporaries become bottlenecks
NumPy / BLAS	Algebraic rewrite, vectorized arrays, matrix identities	Broadcasting can create giant $O(N \cdot K \cdot d)$ or $O(R \cdot n)$ arrays
Numba	Compile scalar numerical loops on CPU while keeping array logic explicit	Compiler restrictions and parallel RNG/correctness details
Threads / Python 3.14	Repeated work over shared arrays; avoid process copies when thread-friendly	Actual GIL state and extension-module behavior must be logged
JAX / A100	Large batched array programs after reformulation, especially matrix-shaped work	Compilation, transfer, device sync, and dtype/device labels

k-means and permutation tests are two canonical statistical computing shapes

ITERATIVE MODEL FITTING

k-means

Statistical question: do we recover known clusters as geometry gets harder?

Compute pressure: assignment/update loops, distance computation, repeated iterations, temporary arrays.

Tool link: Numba for CPU loops; NumPy matmul for distance algebra; JAX/A100 only when the loop becomes large and regular enough.

RESAMPLING INFERENCE

Permutation test

Statistical question: does the null distribution stay calibrated across many features?

Compute pressure: thousands of independent label shuffles across a samples-by-features matrix.

Tool link: threads/processes for repetition; Numba for CPU kernels; JAX/A100 after rewriting the statistic as `w @ x`.

These are not chosen because they are exotic. They are chosen because many statistical workloads look like either iterative dependence or repeated simulation.

Different machines answer different questions

MacBook Air

Small, controlled simulation.
Debugging. Reference equivalence.
Developer experience.



Linux server CPU

Large shape. Parallelism.
Threads/processes. Memory
accounting and worker sweeps.



A100 GPU

Only after the computation
becomes batched, array-shaped,
and device-resident enough to
matter.

Benchmark contract

- Correctness status before runtime.
- Cold first call and warm median.
- Memory, including child RSS for processes.
- Explicit unavailable rows for GPU/JIT/free-threaded tests.

JAX / GPU timing contract

- Always synchronize device work before timing.
- Label dtype and device.
- Separate end-to-end pipeline from kernel-only hypotheses.
- Do not report hoped-for speedups as measured results.

WORKLOAD 1

k-means as iterative simulation pressure

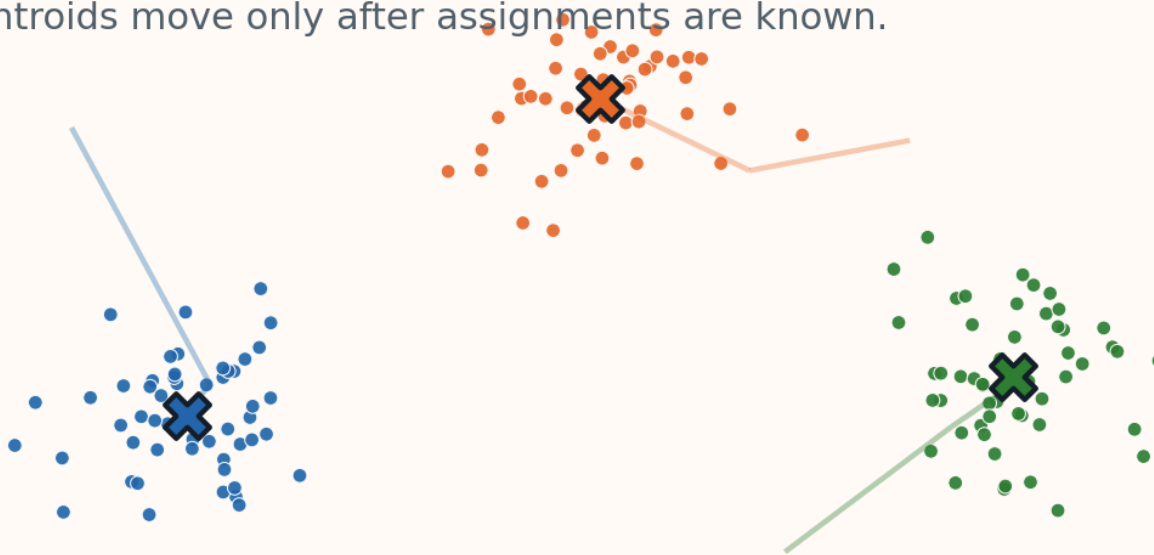
Same simulated data, same initialization, same statistic. Then vary separation, dimension, imbalance, outliers, K, N, and seeds.

How k-means moves

Assignment and centroid update form an iterative dependency loop.

Iteration 5: assign points

Centroids move only after assignments are known.



One Lloyd loop

1. assign each point to the nearest centroid
2. update centroids from assigned points
3. repeat until stable

Each iteration depends on the previous centroids.

This is why explicit loops, temporary arrays, and distance computation connect naturally to Numba, NumPy matmul, and JAX/A100.

A simple algorithm exposes the loop patterns in many statistical methods

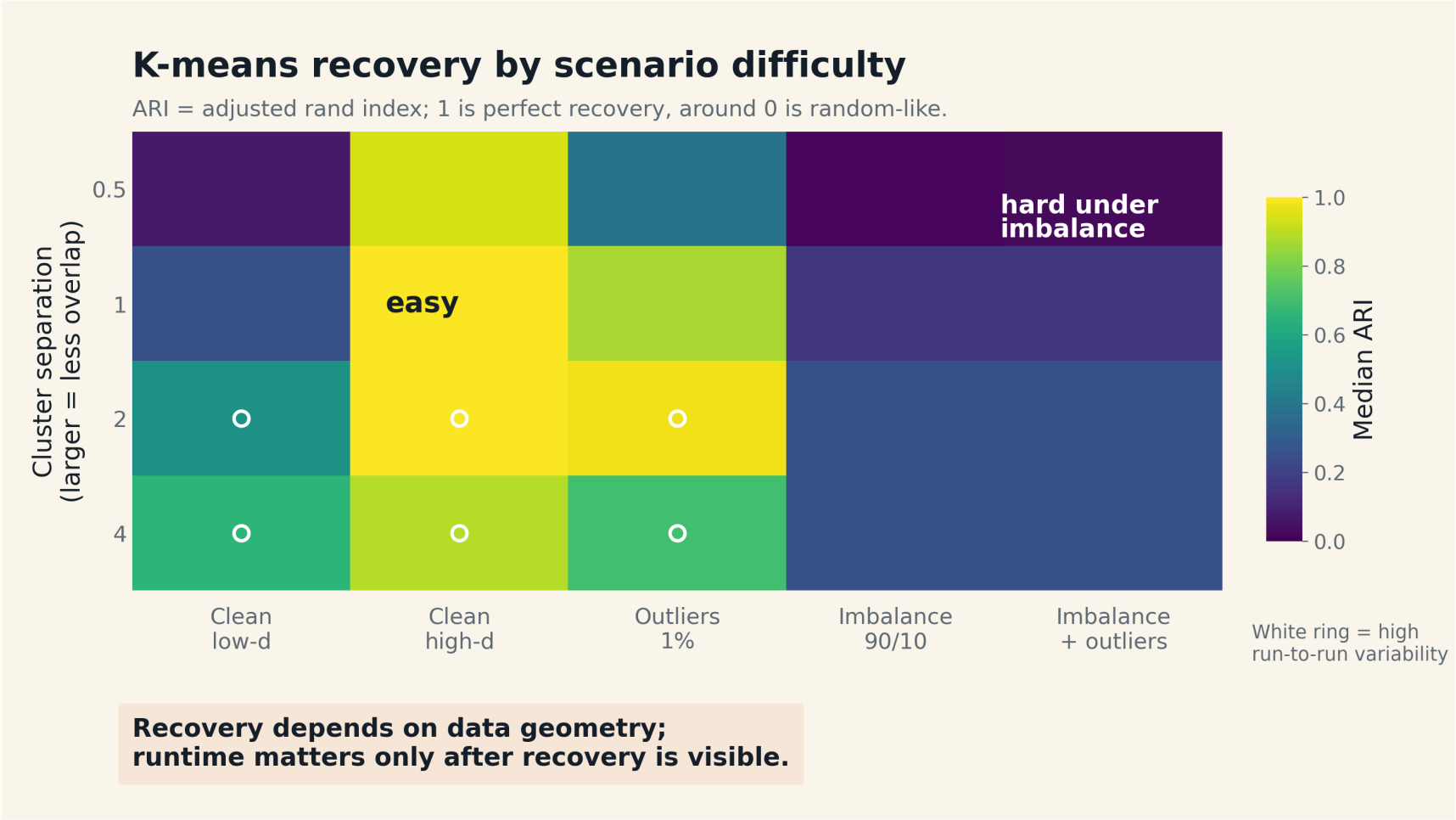
Statistical target
Recover known clusters and preserve fixed-initialization inertia across implementations.

Failure modes
Low separation, high dimension, imbalanced clusters, outliers, bad initialization, empty clusters.

Python pressure
Interpreter loops, $O(N \cdot K \cdot d)$ broadcast temporaries, BLAS-friendly distance identities, compiled loop kernels.

IMPLEMENTATION IDEA	WHY IT IS RELEVANT	WHAT SIMULATION CHECKS
Readable NumPy	Closest to the math; easiest to debug	Reference inertia and recovery
Numba loop	Compile assignment/update loops without giant temporary arrays	Same fixed-init result, faster CPU loop
NumPy matmul	Use distance algebra to move work into BLAS	Same statistic, better shape for large d/K
JAX/A100	Large regular loops can become accelerator work	Device result matches reference before speed claims

Recovery surfaces tell us which runtimes are meaningful

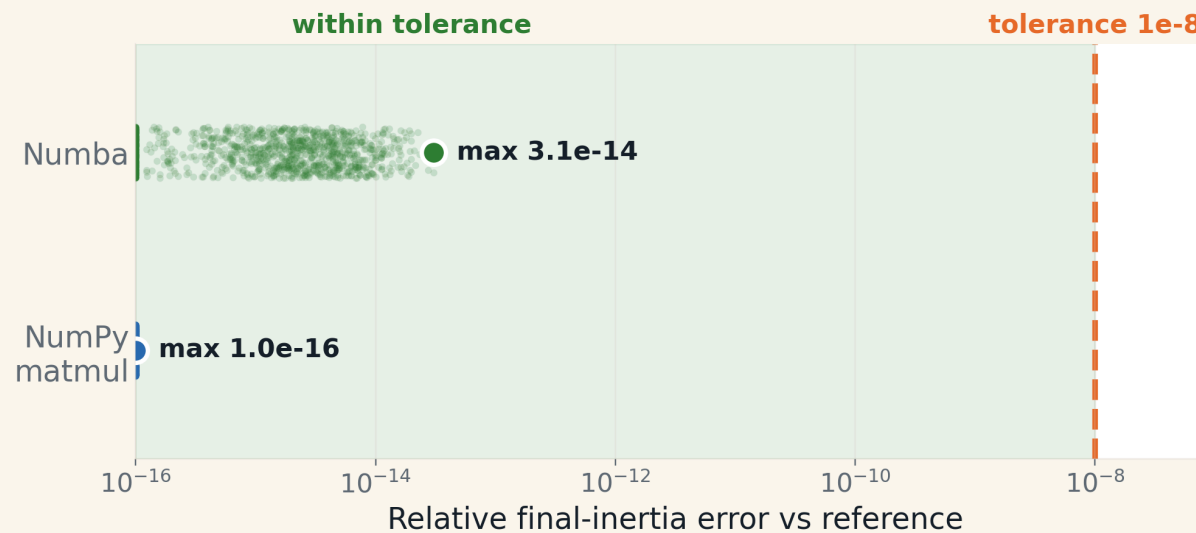


MacBook validation tier: cells show median ARI from existing measured rows; white rings mark high run-to-run variability.

Optimized paths must preserve the reference statistic

K-means equivalence to reference

All optimized rows are inside the $1e-8$ tolerance band. Largest observed relative error: $3.1e-14$.



3,840 pass rows; 480 expected memory-risk skips; no failed optimized rows.

Optimized paths preserve the reference solution within tolerance.

Same initialization and same reference solution; optimized rows are counted only after the tolerance check passes.

On the server, the winner depends on computational shape

Numba wins many CPU loop shapes

Especially when scalar assignment/update loops dominate.

BLAS helps different shapes

Matmul can be attractive when distance algebra matches dense linear algebra.

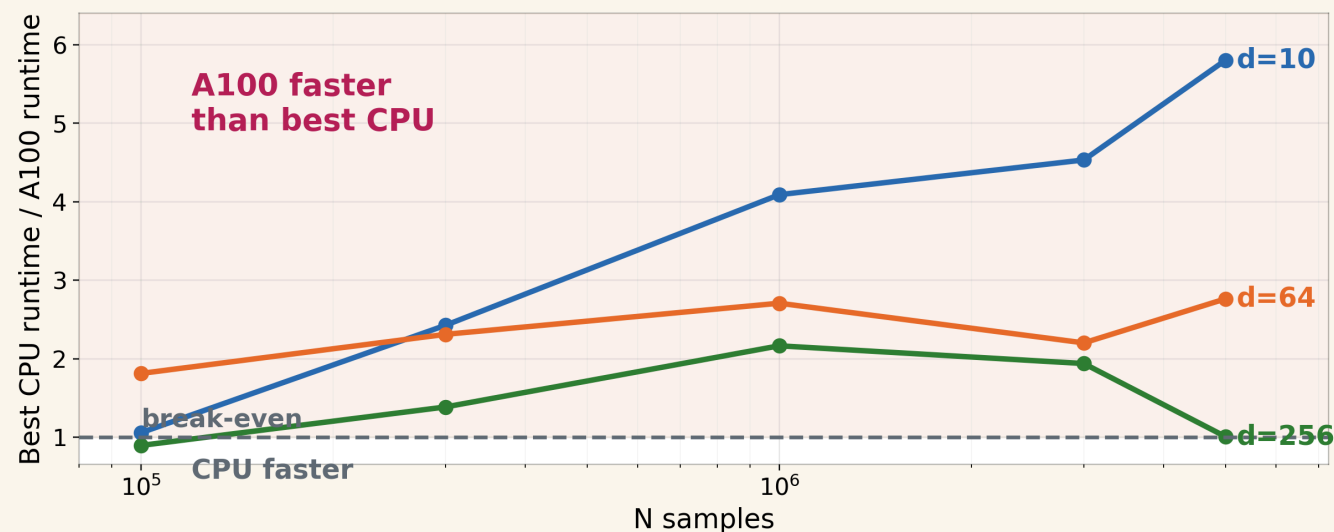
A100 is conditional

It wins only when enough regular work exists to amortize compilation, transfer, and device overhead.

Server k-means: best CPU baseline vs A100

k=20, separation=2.0; maximum observed A100 advantage 5.8x.

A100 becomes worthwhile only for some k-means shapes



A100 helps only after enough regular work exists.

Measured server rows only: best warm CPU baseline versus A100, k=20 and separation=2.0; max matched A100 edge 5.8x.

The tool choice follows the simulation signal

SIGNAL FROM THE EXPERIMENT	LIKELY TOOL	REASON
Reference behavior not stable	Do not optimize yet	The statistical contract is not ready.
Small K/d, scalar loops dominate	Numba	Fuses explicit loops with low extra memory.
Large K/d and dense distance work	NumPy matmul / BLAS	Algebraic rewrite can remove expensive broadcast temporaries.
Huge N and regular array program	JAX/A100	Potential accelerator win after enough work exists.
Poor ARI under simulated truth	Revise method or scenario	Runtime is not the first question.

k-means stands in for a large class of iterative statistical algorithms: EM, coordinate descent, optimization loops, and simulation-based estimators.

WORKLOAD 2

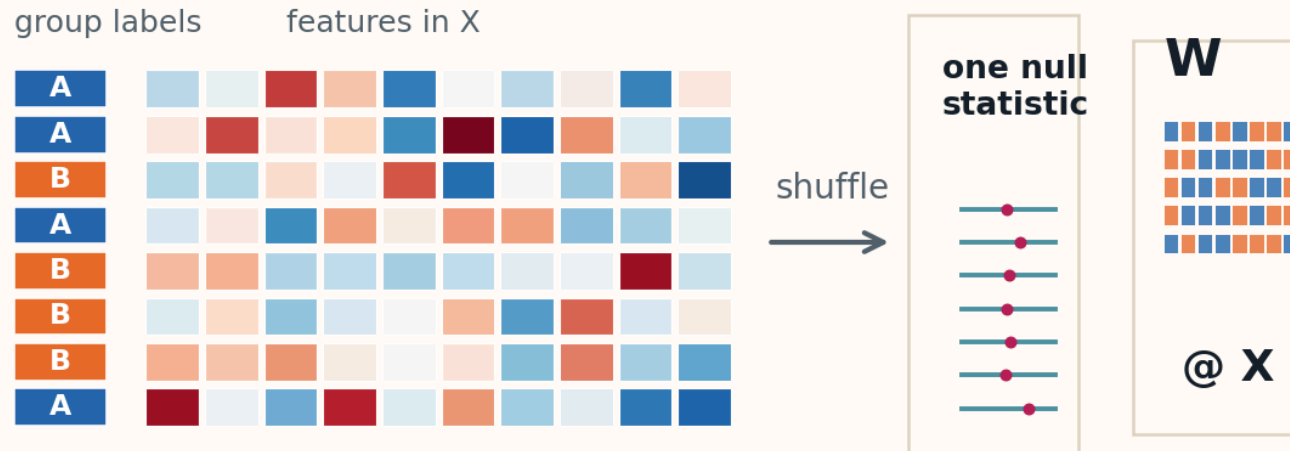
Permutation tests as resampling pressure

A simple inference procedure becomes expensive when we need thousands of null draws across a high-dimensional features matrix.

How a permutation test scales

One simple statistic becomes many independent null draws.

Shuffle labels -> statistic -> repeat



One resampling loop

1. shuffle group labels
2. recompute the statistic
3. repeat many times to form the null distribution

The repetitions are independent, but memory, random generation, and batching decide performance.

This is why permutation tests connect to threads/processes, Numba kernels, and JAX/A100 matrix formulations.

They expose the other common shape: independent statistical repetition

Biostatistics context

Feature-wise testing appears in gene expression, imaging, biomarkers, and other high-dimensional settings.

Statistical contract

Under the null, p-values should be calibrated; optimized p-values must match the reference using the same permutations.

Python pressure

Repeated random index work, shared arrays, process copies, thread ceilings, matrix batching, and GPU transfer overhead.

For programmers: this is the workload where "embarrassingly parallel" still has hard engineering details.

Same statistic, different computational formulation

Readable reference loop

```
perms = make_permutations(labels, R)

for r in range(R):
    stat[r] = diff_mean(X, perms[r])
```

Matrix formulation

```
perms = make_permutations(labels, R)
W = contrast_matrix(perms)
stat = W @ X
```

The rewrite is allowed only because both paths are checked against the same permutations. Otherwise differences may be random-number differences, not implementation differences.

First, prove the matrix path matches the reference

Same permutation stream; same p-values; only floating-point-scale statistic gaps.

Same permutations

Reference loop and matrix path use the same label draws.

Same p-values

$\max |p \text{ diff}| = 0.0$ across checked rows.

Floating-point statistic gap

$\max |\text{stat diff}| = 9.4\text{e-}16$, far below tolerance $1\text{e-}6$.

Same statistic, different computational shape.

PASS

18 workloads × 5 seeds
same permutation stream
JAX tier: CPU/x64, not A100

450

pass rows admitted
to comparison

0

failed rows

45

expected memory-
risk skips

Nonzero statistic differences only; tolerance $1\text{e-}6$

NumPy matrix statistics
 $9.4\text{e-}16$



JAX CPU/x64 matrix
statistics
 $8.9\text{e-}16$



1e-18 1e-15 1e-12 1e-9 **1e-6**

Dashed line: tolerance $1\text{e-}6$.

Then check null calibration

Under the null, the rejection rate should stay close to $\alpha = 0.05$.

Why calibration matters

Equivalence checks implementation. Calibration checks statistical behavior.

Observed check

At $\alpha = 0.05$, observed type-I rate = 0.051.

What this unlocks

Now runtime comparisons become meaningful.

PASS

0.051

mean replicate-level proportion with $p \leq 0.05$

0.050

nominal alpha

100

null replicates

500

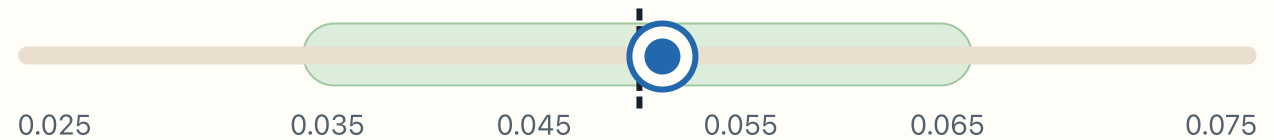
n

1,000

p and R

Calibration ruler

expected Monte Carlo band 0.036-0.064



Dashed: nominal 0.05

Dot: observed mean 0.051

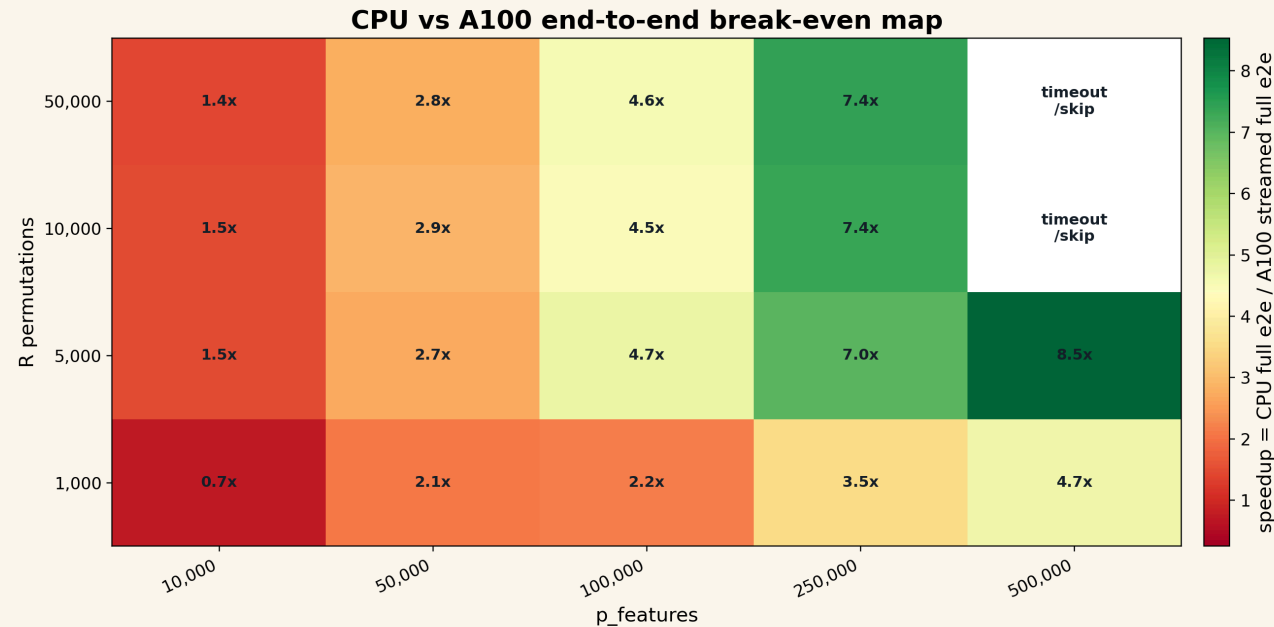
Expected band: $0.05 \pm 1.96 * \sqrt{0.05 * 0.95 / 1000}$.

Fast code is useful only if null calibration remains correct.

DECISION MAP

When does GPU help for permutation tests?

A100 wins only after the permutation pipeline becomes large, batched, and device-resident enough.



A100 faster region appears by $n=5000$, $p=10,000$, $R=5,000$, $\text{batch_R}=8,192$.

CPU wins when

p and R are small or random generation, W construction, transfer, collection, or too-small batch_R dominates.

A100 can win when

R x p is large, batch_R fits memory, streamed reduction avoids collecting R x p, and work stays on device.

Measured scope

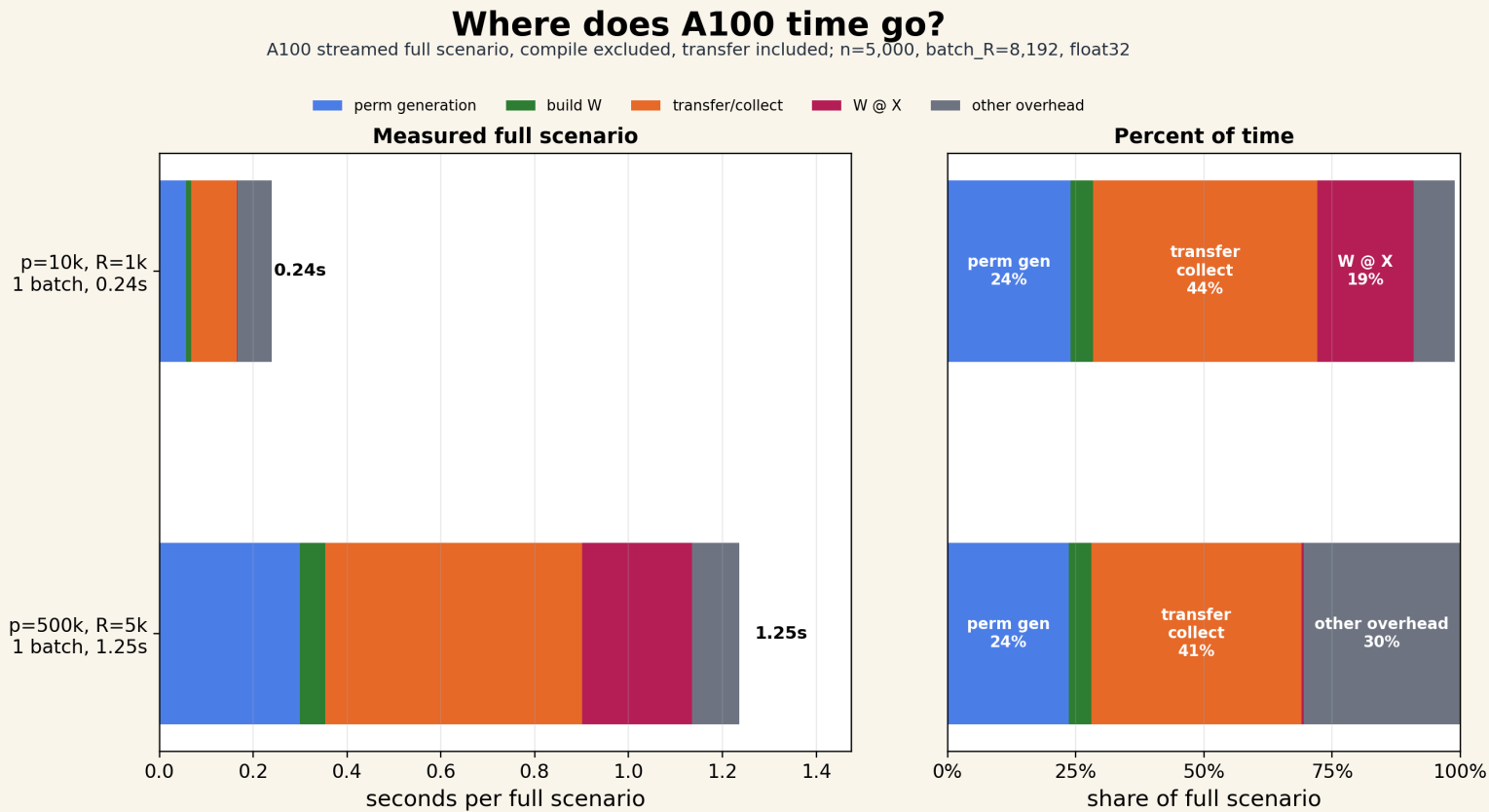
$n=5,000$ map; $\text{batch_R}=8,192$.
First measured win at $p=10,000$, $R=5,000$; largest measured speedup 8.54x.

Speedup = matched CPU matrix baseline / A100 streamed full end-to-end; compile excluded, transfer included, kernel-only excluded.

Shape beats logo: GPU is a formulation decision, not a flag.

Where does A100 time go?

A fast matrix multiply is not enough; the full statistical pipeline decides CPU vs GPU.



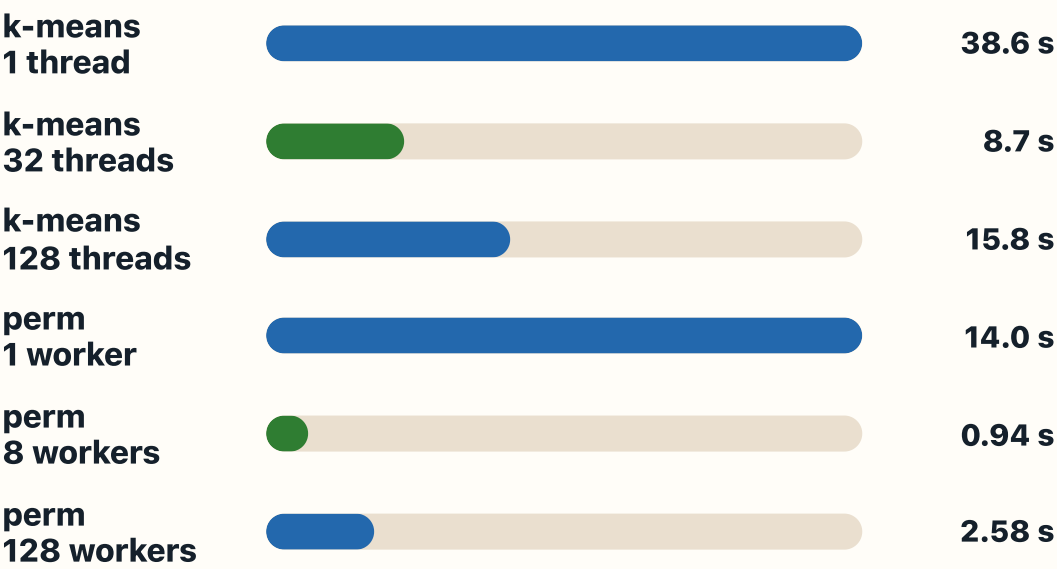
Full scenario timing; compile excluded, transfer included; streamed reduction; batch_R=8,192; n_batches shown per scenario; residual labeled as other overhead.

More workers is not a moral victory

Best speed is not max workers; memory can keep growing after runtime saturates.

Runtime improves, then turns

Measured server CPU sweeps; lower is better.



Memory keeps a budget attached

Peak host memory from the same sweeps.



Measure the stack you actually run: thread counts, worker counts, CPU affinity, and memory pressure.

Python 3.14, Numba, and JAX solve different parts of the workflow

Python 3.14

Changes interpreter/runtime ceilings: free-threaded builds can help thread-friendly workloads; experimental JIT rows must be reported only when actually available and enabled.

Numba

Best fit when simulation identifies explicit numerical loops over arrays as the hotspot, especially CPU kernels where avoiding temporaries matters.

JAX / A100

Best fit when the statistic has been reformulated as a large batched array program and device execution can dominate transfer and compilation overhead.

The decision is not Python versus compiled languages. It is: keep Python as the scientific interface, then accelerate the pattern that simulation proves is expensive.

Codex can generate variants; the statistician owns the contract

Good Codex work

- Generate implementation variants.
- Build scenario-grid runners.
- Record environment metadata.
- Regenerate plots from CSVs.
- Keep result paths and READMEs consistent.

Human-owned decisions

- Define the statistic and null model.
- Choose scenario grids and failure criteria.
- Decide whether skips are honest.
- Interpret calibration, recovery, and negative speed evidence.

AI is a copilot for code generation and experiment hygiene, not an oracle for statistical truth.

Choose the smallest tool that preserves the statistic

IF THE BOTTLENECK IS...	PREFER...	WHY
Understanding or correctness	Python / NumPy reference	Debuggable, inspectable, close to the math.
Scalar CPU loops	Numba	High return without changing the algorithmic shape.
Repeated work over shared arrays	Threads / free-threaded Python / Numba	Avoid process copies; measure the actual runtime ceiling.
Process-isolated jobs	multiprocessing	Robust, but memory and startup cost need budgeting.
Large batched array programs	JAX / GPU	Accelerator-friendly only after reformulation.
Giant temporaries	Algebraic rewrite	Often safer than switching libraries blindly.

CLOSE

Make the statistic testable, then make the bottleneck fast.



Clear enough to trust. Fast enough to scale.

`github.com/lucajiang/FastStatisticalModels4Python`

Evidence map: keep tiers separate

MacBook Air trust tier

k-means equivalence, recovery surfaces, permutation p-value equivalence, calibration, and power.

Linux server CPU scale tier

k-means shape scaling, Numba thread sweep, permutation R/p scaling, worker and memory sweeps.

A100 accelerator tier

JAX k-means break-even, old permutation matched-slice negative evidence, and the streamed-reduction break-even follow-up.

3,840

k-means local pass rows

450

permutation equivalence
pass rows

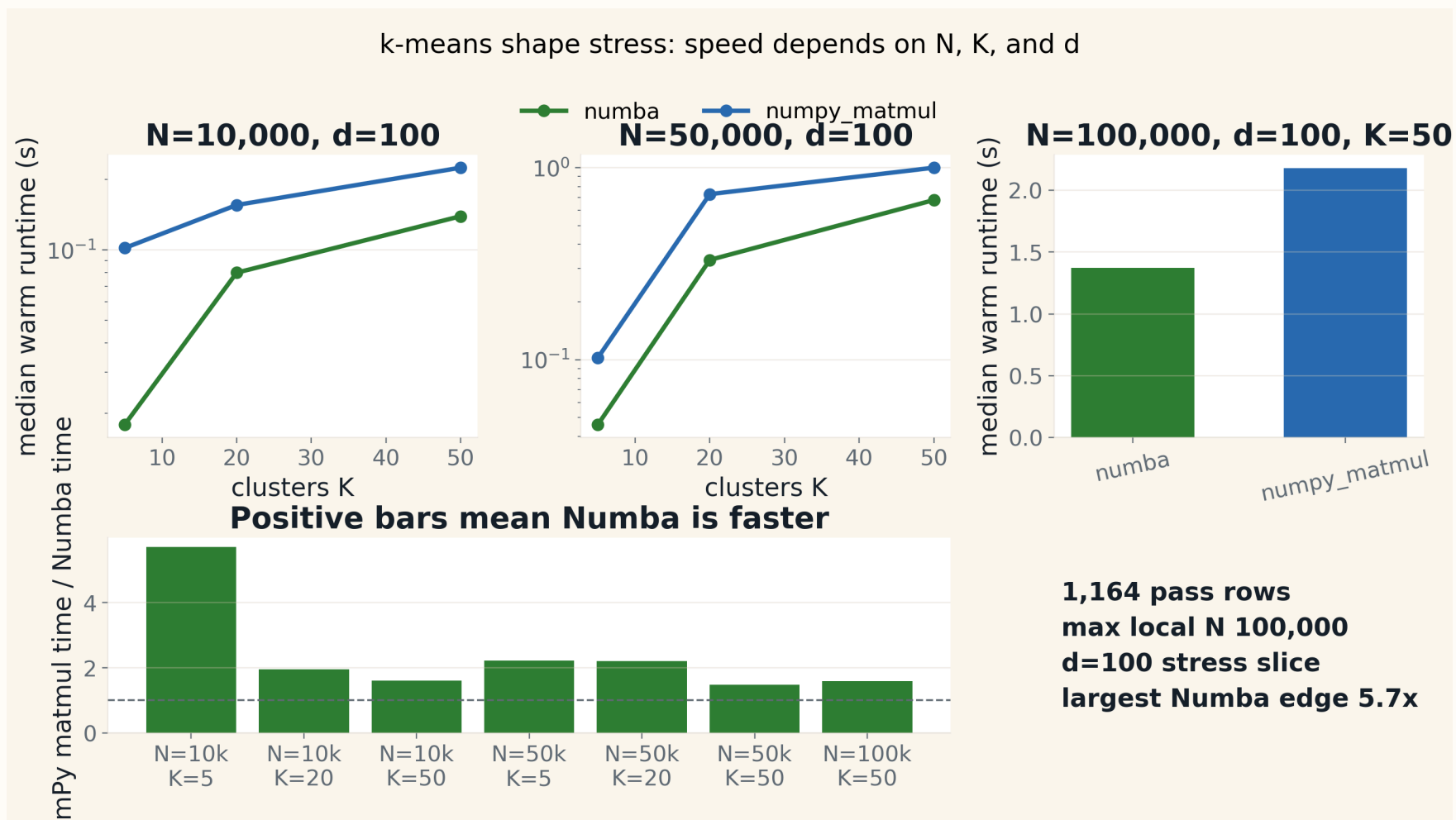
540

server CPU k-means pass
rows

135

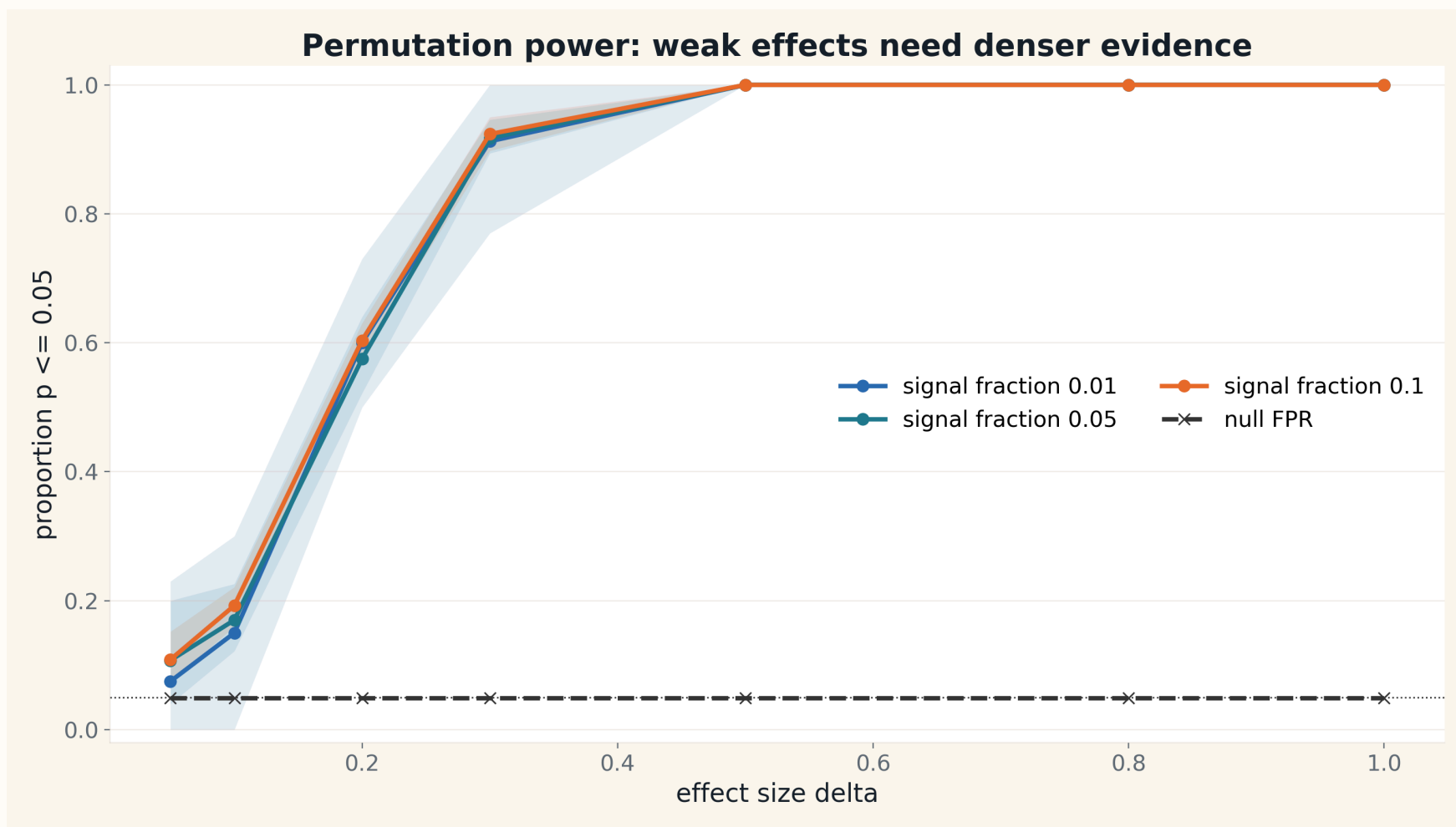
A100 k-means pass rows

Shape stress: K and d become the bottleneck



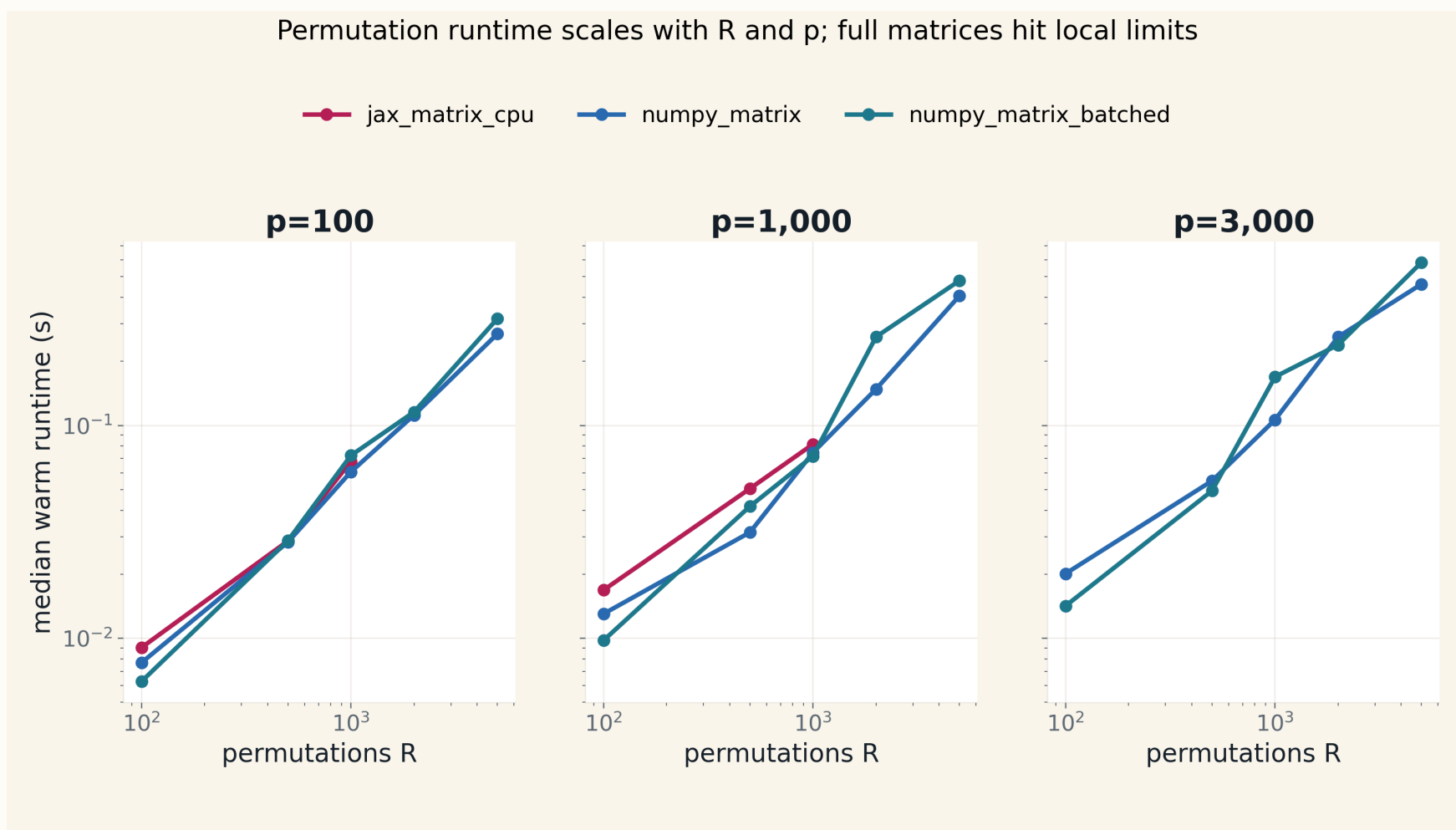
Extra MacBook evidence: 1,164 shape-stress rows, all pass.

Permutation power rises with effect size



Backup statistical sanity check: weak effects need denser evidence; larger effects reach high power in this local curve.

Local permutation runtime follows computational shape



MacBook CPU only: useful for local validation, not a server or GPU leaderboard.

What the agent actually changed

Experiment surface

Replaced quick outputs with resumable MacBook evidence, server long-safe CSVs, and result READMEs.

Visualization surface

Regenerated figures from CSVs and removed charts that looked precise but were unreadable.

Repository surface

Updated documentation so future agents land on the current simulation contract.